Week 4 Studio: Validating Form with HTML and JavaScript

Overview: Content

Welcome to **the fourth week** of your Vue.js 3 learning journey!

This week, we will explore how to validate forms using HTML and JavaScript. You'll learn how to leverage **built-in HTML validation** features and implement custom validation with **JavaScript** to ensure your forms are user-friendly and reliable. In addition, we will introduce **PrimeVue**, a complete UI suite for Vue.js consisting of a rich set of UI components, icons, blocks, and application templates. By the end of this studio, you'll have an understanding of form validation techniques and the PrimeVue library.

Here's what we'll cover:

- **HTML Built-in Form Validation:** Learn how to use HTML form validation features, which generally don't require much JavaScript and perform better than JavaScript-based validation, though it is not as customizable.
- **JavaScript Validation:** Discover how to do form validation using JavaScript, which is completely customizable and allows developers to tailor the validation to specific needs.
- **Introduction to PrimeVue:** Introduce PrimeVue, a UI component library for Vue.js. And explore how to integrate PrimeVue into your Vue project and how to use its components to enhance a web application.



Dont worry if you don't finish everything in this Studio during the 2-hour lab - as long as you finish everything by the end of the week (**Friday 11:55pm**), you are doing great!

Overview: Lab tasks for eFolio

TOPICS

- HTML Built-in and JavaScript Form Validations
- PrimeVue DataTable

INSTRUCTIONS

- Form a group of 3-4 students.
- Collaborate and engage with each other.

> ✗ While you can freely help each other, be careful when sharing code in any assessed labs with someone else. This might be considered **collusion**.

SUBMISSION

- Submit eFolio tasks 3 & 4 together on Moodle.

EFOLIO TASK 4.1 (PASS AND CREDIT LEVEL)

DESCRIPTION:

- Understand and validate a form with HTML Built-in and JavaScript Form Validations.
- Display a form in a VueJS app with prescribed validations.

WHAT TO SUBMIT (INDIVIDUAL):

- **Screenshot 1**: Submit screenshots of a VueJS app with validation enabled, demonstrating the completion of Activities 1-3.

> ✓ You should have multiple screenshots to show each activity separately.

EVALUATION CRITERIA:

- All activities are completed properly.

EFOLIO TASK 4.2 (DISTINCTION AND HIGH DISTINCTION LEVEL)

DESCRIPTION:

- Extend the project in eFolio 4.1 (Validation enabled VueJS app)
- Add extra validations (except those from Activities 1-3), bringing the total number of validations to five.
- Use PrimeVue DataTable to display submitted user information.
- Push all your code to the GitHub for management and maintenance. This needs to be a private repo, shared with your tutor.

WHAT TO SUBMIT (INDIVIDUAL):

- screenshots of your front end demonstrating the form support on extra validations.
- a screenshot of your front end demonstrating a PrimeVue DataTable with submitted user information.
- a screenshot of the code for implementing extra validations and PrimeVue DataTable.
- a screenshot showing the GitHub commit history with your project update.

See this article about how to see the commit history of a Github repo: https://docs.github.com/en/pull-requests/committing-changes-to-your-project/viewing-and-comparing-commits/differences-between-commit-views

EVALUATION CRITERIA:

- the User Information Form supports extra validations.
- a PrimeVue DataTable displays submitted user information.
- GitHub history with the successful push.

Pre-lab: Introduction to HTML Built-in and JavaScript Form Validations (10 mins)

From our pre-reading, you should have a basic understanding of HTML Built-in and JavaScript form validations. If you haven't finished the pre-reading, please read it first!

Now, let's dive deeper into the details of these validation methods.

HTML built-in form validation

▼ 扩大

One of the most significant features of modern form controls is the ability to validate most user data without relying on JavaScript. This is done by using validation attributes on form elements including:

- `required`: Specifies whether a form field needs to be filled in before the form can be submitted.
- `minlength` and `maxlength`: Specifies the minimum and maximum length of textual data (strings).
- `min` and `max`: Specifies the minimum and maximum values of numerical input types.
- `type`: Specifies whether the data needs to be a number, an email address, or some other specific preset type.
- `pattern`: Specifies a regular expression that defines a pattern the entered data needs to follow.

If the data entered in a form field follows all of the rules specified by the above attributes, it is considered valid. If not, it is considered invalid.

When an element is valid, the following things are true:

- The element matches the `:valid` CSS pseudo-class, which lets you apply a specific style to valid elements.
- If the user tries to send the data, the browser will submit the form, provided there is nothing else stopping it from doing so (e.g., JavaScript).

When an element is invalid, the following things are true:

- The element matches the `:invalid` CSS pseudo-class, and sometimes other UI pseudo-classes (e.g., `:out-of-range`) depending on the error, which lets you apply a specific style to invalid elements.
- If the user tries to send the data, the browser will block the form and display an error

message.

Note: Several errors will prevent the form from being submitted, including a `badInput`, `patternMismatch`, `rangeOverflow` or `rangeUnderflow`, `stepMismatch`, `tooLong` or `tooShort`, `typeMismatch`, `valueMissing`, or a `customError`.

JavaScript Form Validation

▼ 扩大

JavaScript is used if you want to take control over the look and feel of native error messages. Normally, developers are following these steps:

- **Data Retrieval:** It retrieves the values of various form fields like name, address, email, and password.
- **Data Validation:**
  - **Name Validation:** Checks if the name field is empty or contains any digits.
  - **Address Validation:** Ensures the address field is not empty.
  - **Email Validation:** Verifies if the email field is not empty and contains the '@' symbol.
  - **Password Validation:** Validates that the password field is not empty and has a minimum or maximum length.
- **Error Handling:**
  - If any of the validation criteria fail, it displays an alert message.
  - Sets focus back to the respective field that failed validation, ensuring the user's attention is drawn to the problematic field.
- **Submission Control:**
  - Returns true if all validation checks pass, indicating that the form can be submitted. Otherwise, it returns false, preventing form submission.
- **Focus Adjustment:**
  - Sets focus on the first field that failed validation, ensuring the user's attention is drawn to the problematic field.

Step 1: Lab Set-up (10 mins)

Last week, we added the **user information form** to our library web application to gather user information and make it user-friendly with the help of Bootstrap 5. However, our form currently lacks constraints on user input, allowing users to submit anything they want. This may lead to incorrect information and, in some cases, malicious inputs that could harm our application. Therefore, it is essential to validate user input before submission to ensure:

- **Data Accuracy:** By validating user input, we ensure that the information collected is accurate and reliable. This accuracy is crucial for providing a personalized user experience. For instance, if we gather correct user preferences and demographic details, we can recommend books and authors that align with their interests, making the application more useful and engaging for them.
- **Security:** Validating form inputs helps protect the application from malicious inputs that could potentially exploit vulnerabilities and compromise the system. For example, without validation, a user could enter a script or SQL injection that might harm our database or expose sensitive information. Implementing proper validation ensures that only safe and expected data is processed, safeguarding the application's integrity and user data.
- **User Experience:** Providing immediate feedback on user inputs enhances the overall user experience. When users make errors, such as leaving required fields blank or entering data in an incorrect format, validation can prompt them to correct these mistakes in real time. This not only prevents frustration but also makes the form easier and more intuitive to complete. A smooth and responsive form increases user satisfaction and encourages them to provide the necessary information accurately.

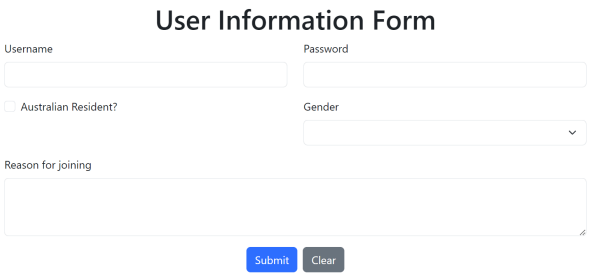
Now, let's get started with adding validation to our **user information form**.

Activity 1: Run Form Vue component

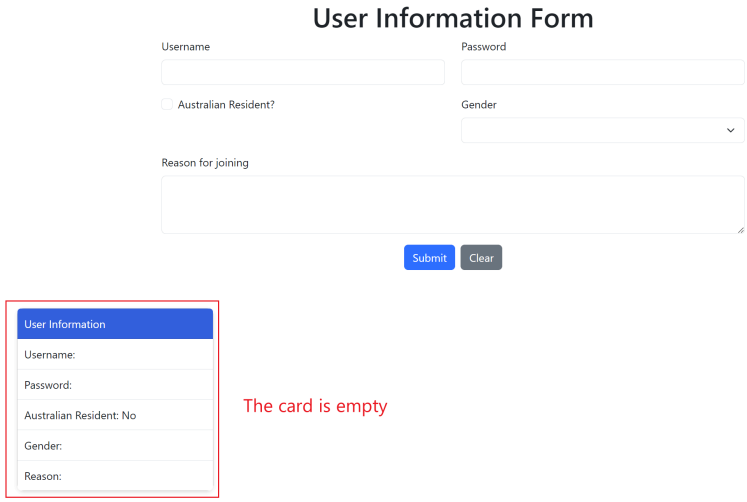
1.1 Copy or clone the library web application from week 3 to week 4, and run it with the following code:

```
npm run dev
```

1.2 Now we can see the user information form as shown below:



1.3 Without modification, click the **Submit** button directly and the page will show as below:



Here, we can see the user can submit an empty form without any validation.

In the next activity, we will add our first validation to the form using HTML built-in validation.

Step 2: Validate User Input with HTML Built-in Form Validation (30 mins)

Activity 2: Add Form Validation with HTML Built-in Attributes

2.1 Open `LibraryRegistrationForm.vue/Form.vue` and add the first validation to the input field "Username" using the `required` attribute, as shown below:

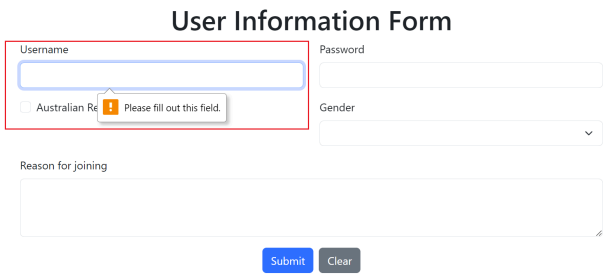
> ⚠ You might notice some screenshots or content in the slides display "Form.vue" instead of "LibraryRegistrationForm.vue". This is just a naming difference to avoid an ESList error where a component should always be multi-word. The functionality of the component stays the same.

Form.vue | App.vue

src > components > Form.vue > {} template > div.container.mt-5 > div.row > div.col-md-8.offset-md-2 > form > div.row.mb-3 > div.col-md-6.col-

```vue
<template>
  <div class="container mt-5">
    <div class="row">
      <div class="col-md-8 offset-md-2">
        <h1 class="text-center">User Information Form</h1>
        <form @submit.prevent="submitForm">
          <div class="row mb-3">
            <div class="col-md-6 col-sm-6">
              <label for="username" class="form-label">Username</label>
              <input type="text" class="form-control" id="username" required v-model="formData.username" />
            </div>
            <div class="col-md-6 col-sm-6">
              <label for="password" class="form-label">Password</label>
              <input type="password" class="form-control" id="password" v-model="formData.password" />
            </div>
          </div>
          <div class="row mb-3">
            <div class="col-md-6 col-sm-6">
              <div class="form-check">
                <input type="checkbox" class="form-check-input" id="isAustralian" v-model="formData.isAustralian" />
                <label class="form-check-label" for="isAustralian">Australian Resident?</label>
              </div>
            </div>
```

2.2 Go back to our browser and click the **Submit** button again, the page will show as below:



Now we can see that there is no card displayed under the form as It requires that the “Username“ field cannot be empty.

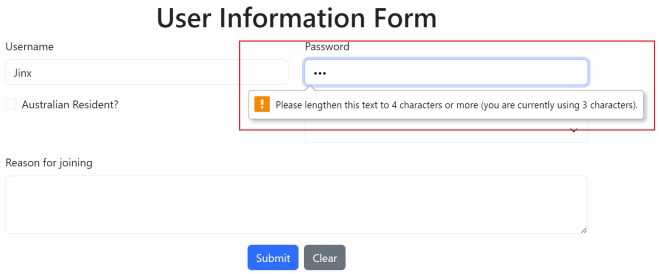
2.3 Let's add another validation to the "Password" input field using `minlength` and `maxlength` attributes, as shown below:

Form.vue | App.vue

src > components > Form.vue > { } template > div.container.mt-5 > div.row > div.col-md-8.offset-md-2 > form > div.row.mb-3 > div.col-md-6.col-sm-6 > label.form-label

```
<template>
  <div class="container mt-5">
    <div class="row">
      <div class="col-md-8 offset-md-2">
        <h1 class="text-center">User Information Form</h1>
        <form @submit.prevent="submitForm">
          <div class="row mb-3">
            <div class="col-md-6 col-sm-6">
              <label for="username" class="form-label">Username</label>
              <input type="text" class="form-control" id="username" required v-model="formData.username" />
            </div>
            <div class="col-md-6 col-sm-6">
              <label for="password" class="form-label">Password</label>
              <input type="password" class="form-control" id="password" minlength="4" maxlength="10" v-model="formData.password" />
            </div>
          </div>
```

2.4 Back to the browser, type a username with **password "123"** and click the Submit button, the page will show as below:



Since our inputted password is **less than 4 digits**, the form thus prevents the submission and displays an error message to inform the user.

2.5 Now, it is time to practice by yourself. Try to add validations to the rest of the input fields including "Australian Resident?", "Gender" and "Reason for joining".

Step 3: Form Validation using Vue (40 mins)

While HTML forms provide basic validation, they have several limitations:

- **Limited Validation Rules:** The built-in validation rules in HTML are basic and cover only common scenarios such as required fields, minimum and maximum lengths, and simple patterns. However, they lack the flexibility to handle more complex validation logic that might be needed for specific applications.
- **No Custom Styling for Error Messages:** HTML form validation uses browser default styles for displaying error messages. This means you cannot easily customize the appearance of these messages to match the design of your web application, leading to a less cohesive user experience.
- **Restricted Validation Behavior:** The behavior of built-in validation is limited. For example, you cannot control when error messages are displayed— they typically appear when the user attempts to submit the form. Additionally, you cannot easily customize the messages themselves, which are predefined by the browser and might not provide the best user guidance.
- **Full Page Refresh on Submission:** When a form is submitted with built-in HTML validation, it typically triggers a full page refresh. This can be disruptive to the user experience, especially in modern web applications where seamless, dynamic interactions are expected.

Now, let's implement our form validation with Vue!

Activity 3: Add Form Validation using Vue

3.1 Remove all HTML validations added to the form before in Activity 2, as shown below:

```vue
<h1 class="text-center">User Information Form</h1>
<form @submit.prevent="submitForm">
  <div class="row mb-3">
    <div class="col-md-6 col-sm-6">
      <label for="username" class="form-label">Username</label>
      <input type="text" class="form-control" id="username" v-model="formData.username" />
    </div>
    <div class="col-md-6 col-sm-6">
      <label for="password" class="form-label">Password</label>
      <input type="password" class="form-control" id="password" v-model="formData.password" />
    </div>
  </div>
  <div class="row mb-3">
    <div class="col-md-6 col-sm-6">
      <div class="form-check">
        <input type="checkbox" class="form-check-input" id="isAustralian" v-model="formData.isAustralian" />
        <label class="form-check-label" for="isAustralian">Australian Resident?</label>
      </div>
    </div>
    <div class="col-md-6 col-sm-6">
      <label for="gender" class="form-label">Gender</label>
      <select class="form-select" id="gender" v-model="formData.gender">
        <option value="male">Male</option>
        <option value="female">Female</option>
        <option value="other">Other</option>
      </select>
    </div>
  </div>
  <div class="mb-3">
    <label for="reason" class="form-label">Reason for joining</label>
    <textarea class="form-control" id="reason" rows="3" v-model="formData.reason"></textarea>
  </div>
  <div class="text-center">
    <button type="submit" class="btn btn-primary me-2">Submit</button>
    <button type="button" class="btn btn-secondary" @click="clearForm">Clear</button>
  </div>
</form>
```

3.2 In `Form.vue` within the `<script setup> </script>`, let's first define a corresponding object to store all the errors in the form, as shown below:

```vue
<script setup>
import { ref } from 'vue';

const formData = ref({
  username: '',
  password: '',
  isAustralian: false,
  reason: '',
  gender: ''
});

const submittedCards = ref([]);

const submitForm = () => {
  validateName(true);
  if (!errors.value.username) {
    submittedCards.value.push({ ...formData.value });
    clearForm();
  }
};

const clearForm = () => {
  formData.value = {
    username: '',
    password: '',
    isAustralian: false,
    reason: '',
    gender: ''
  };
};

const errors = ref({
  username: null,
  password: null,
  resident: null,
  gender: null,
  reason: null,
});
```

`const errors = ref({ ... })`: We declare a constant named `errors` and assign it a reactive reference using `ref()`. The `ref` function is provided by Vue.js and makes the `errors` object reactive, meaning Vue will track its changes and update the UI accordingly.

`{ username: null, password: null, resident: null, gender: null, reason: null }`: This is an object that contains keys corresponding to each form field we want to validate. Initially, all values are set to `null`, indicating that there are no errors in these fields.

3.3 Under the `errors` object, let's define the first function that checks if the "Username" is valid, as shown below:

```
const errors = ref({
  username: null,
  password: null,
  resident: null,
  gender: null,
  reason: null,
});

const validateName = (blur) => {
  if (formData.value.username.length < 3) {
    if (blur) errors.value.username = "Name must be at least 3 characters";
  } else {
    errors.value.username = null;
  }
};
</script>
```

`const validateName = (blur) => { ... }:` This line declares a function named `validateName` that takes a single parameter `blur`. The `blur` parameter is a boolean that indicates whether the validation was triggered by the user leaving (blurring) the input field.

`if (formData.value.username.length < 3) { ... }:` Inside the function, we check if the length of the username entered by the user is less than 3 characters. `formData.value.username` accesses the current value of the username from our reactive `formData` object.

`if (blur) errors.value.username = "Name must be at least 3 characters";:` If the username is less than 3 characters and the validation was triggered by the user leaving the input field (`blur` is true), we set an appropriate error message in `errors.value.username`. This message will be displayed to inform the user that the username is too short.

`else { errors.value.username = null; }:` If the username is at least 3 characters long, we clear any previous error message by setting `errors.value.username` it to `null`. This indicates that the username is valid.

Please complete the above steps (3.1 - 3.3) before starting step 3.4.

3.4 After defining the `validateName` function, we need to modify our "Username" input field accordingly, as shown below:

```
<form @submit.prevent="submitForm">
  <div class="row mb-3">
    <div class="col-md-6 col-sm-6">
      <label for="username" class="form-label">Username</label>
      <input type="text" class="form-control" id="username"
        @blur="() => validateName(true)"
        @input="() => validateName(false)"
        v-model="formData.username" />
        <div v-if="errors.username" class="text-danger">{{ errors.username }}</div>
    </div>
```

`@blur="() => validateName(true)"`: This is a Vue.js event directive. It listens for the `blur` event, which occurs when the user leaves (unfocuses) the input field. When this event is triggered, it calls the `validateName` function with the argument `true`, indicating that the validation was triggered by a blur event.

`@input="() => validateName(false)"`: This is another Vue.js event directive. It listens for the `input` event, which occurs whenever the value of the input field changes (e.g., when the user types). When this event is triggered, it calls the `validateName` function with the argument `false`, indicating that the validation was triggered by an input event.

`<div v-if="errors.username" class="text-danger">{{ errors.username }}</div>`: This `div` element conditionally displays an error message. The `v-if` directive checks if there is an error message stored in `errors.username`. If there is, it renders this `div` and displays the error message inside it. The `text-danger` class from Bootstrap applies red colouring to the text, indicating an error.

3.5 Finally, we need to add the `validateName` function to the `submitForm` function defined from the week3, as shown below:

```vue
<script setup>
import { ref } from 'vue';

const formData = ref({
  username: '',
  password: '',
  isAustralian: false,
  reason: '',
  gender: ''
});

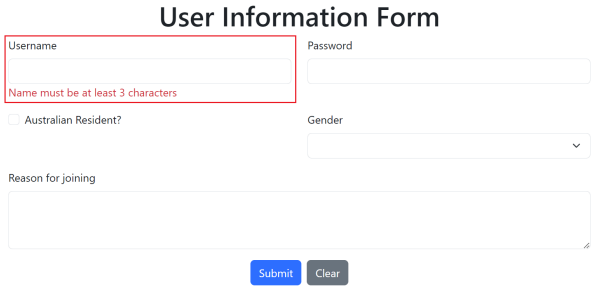
const submittedCards = ref([]);

const submitForm = () => {
  validateName(true);
  if (!errors.value.username) {
    submittedCards.value.push({ ...formData.value });
    clearForm();
  }
};
```

`validateName(true)`: The function starts by calling `validateName` with the argument `true`. This triggers the validation for the username field, specifically indicating that the validation was triggered by a form submission (or a blur event). The `validateName` function checks if the username meets the required criteria (e.g., it must be at least 3 characters long). If the username is invalid, it sets an appropriate error message in `errors.value.username`.

`if (!errors.value.username) { ... }`: This line checks if there is no error message for the username. The condition `!errors.value.username` evaluates to `true` if `errors.value.username` is `null` or an empty string, meaning the username is valid. If the username is invalid (i.e., `errors.value.username` contains an error message), the condition evaluates to `false`, and the block inside the `if` statement is not executed.

3.6 Open the user information form page, **type a username of less than 3 characters** and click the **Submit** button, the page will show as below:



Please complete the above steps (3.1 - 3.6) before starting step 3.7 for other input types.

3.7 Now let's add another validation function to our form input field "Password". As it is very curial, this function will be slightly more complex than the `validateName` function, as shown below:

```
const validateName = (blur) => {
  if (formData.value.username.length < 3) {
    if (blur) errors.value.username = "Name must be at least 3 characters";
  } else {
    errors.value.username = null;
  }
};

const validatePassword = (blur) => {
  const password = formData.value.password;
  const minLength = 8;
  const hasUppercase = /[A-Z]/.test(password);
  const hasLowercase = /[a-z]/.test(password);
  const hasNumber = /\d/.test(password);
  const hasSpecialChar = /[!@#$%^&*(),.?":{}|<>]/.test(password);

  if (password.length < minLength) {
    if (blur) errors.value.password = `Password must be at least ${minLength} characters long.`;
  } else if (!hasUppercase) {
    if (blur) errors.value.password = "Password must contain at least one uppercase letter.";
  } else if (!hasLowercase) {
    if (blur) errors.value.password = "Password must contain at least one lowercase letter.";
  } else if (!hasNumber) {
    if (blur) errors.value.password = "Password must contain at least one number.";
  } else if (!hasSpecialChar) {
    if (blur) errors.value.password = "Password must contain at least one special character.";
  } else {
    errors.value.password = null;
  }
};
</script>
```

`const minLength = 8`: Sets the minimum length requirement for the password.

`const hasUppercase = /[A-Z]/.test(password)`: Checks if the password contains at least one uppercase letter using a regular expression.

`const hasLowercase = /[a-z]/.test(password)`: Checks if the password contains at least one lowercase letter using a regular expression.

`const hasNumber = /\d/.test(password)`: Checks if the password contains at least one number using a regular expression.

`const hasSpecialChar = /[!@#$%^&*(),.?":{}|<>]/.test(password)`: Checks if the password contains at least one special character using a regular expression.

3.8 Similarly, modify our "Password" input field accordingly, as shown below:

```
<form @submit.prevent="submitForm">
  <div class="row mb-3">
    <div class="col-md-6 col-sm-6">
      <label for="username" class="form-label">Username</label>
      <input type="text" class="form-control" id="username"
        @blur="() => validateName(true)"
        @input="() => validateName(false)"
        v-model="formData.username" />
        <div v-if="errors.username" class="text-danger">{{ errors.username }}</div>
    </div>

    <div class="col-md-6 col-sm-6">
      <label for="password" class="form-label">Password</label>
      <input type="password" class="form-control" id="password"
        @blur="() => validatePassword(true)"
        @input="() => validatePassword(false)"
        v-model="formData.password" />
      <div v-if="errors.password" class="text-danger">{{ errors.password }}</div>
    </div>
  </div>
```

3.9 Don't forget to modify the `submitForm` function as well, as shown below:

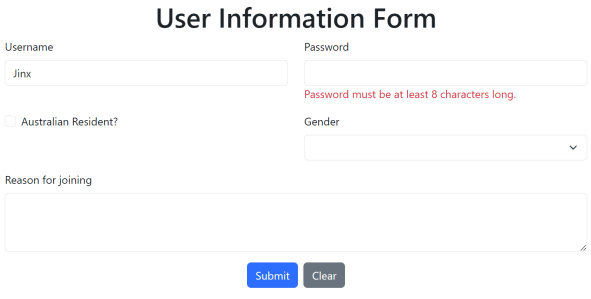
```vue
<script setup>
import { ref } from 'vue';

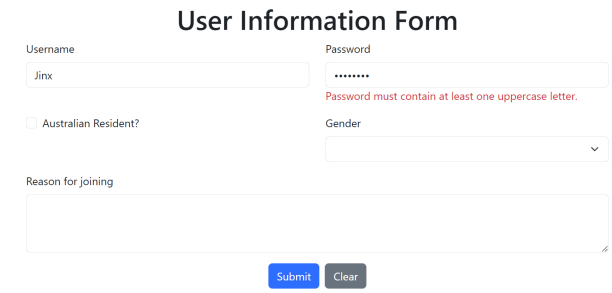
const formData = ref({
  username: '',
  password: '',
  isAustralian: false,
  reason: '',
  gender: ''
});

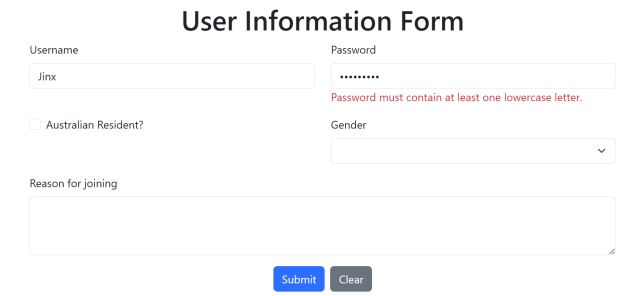
const submittedCards = ref([]);

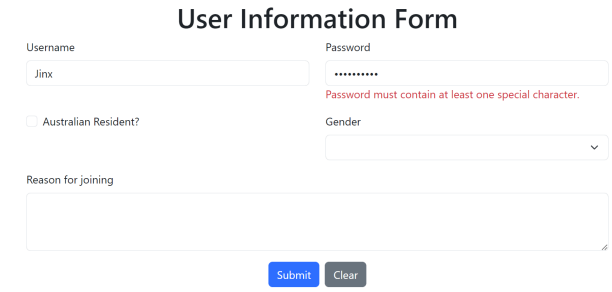
const submitForm = () => {
  validateName(true);
  validatePassword(true)
  if (!errors.value.username && !errors.value.password) {
    submittedCards.value.push({ ...formData.value });
    clearForm();
  }
};
```

3.10 Now, open our user information form page and type a valid username but with a **different invalid password**, the page will look like these:









3.11 Now, it is time to practice by yourself. Try to add validations using **Vue** to the rest of the input fields including "Australian Resident?", "Gender" and "Reason for joining".

⚠ Residency validation could be mandatory in some cases!

[Updated by Russel, August 2026]

Step 4: Display User Information with PrimeVue DataTable (30 mins)

In week 3, we displayed the submitted user information using Bootstrap Cards. While this method helps us view the information users have submitted, it becomes cumbersome when the number of users increases, resulting in a cluttered page with numerous cards. To display such information more efficiently, we will leverage the **PrimeVue DataTable** component.

Introduction to PrimeVue

PrimeVue is a comprehensive UI suite for Vue.js that includes a rich set of UI components, icons, blocks, and application templates. Its primary goal is to boost developer productivity by providing reusable, customizable solutions.

Theming

PrimeVue offers two theming modes:

- **Styled Mode:** Pre-skinned components with predefined themes like Aura, Lara, or Nora.
- **Unstyled Mode:** Provides functionality and accessibility without predefined styles, allowing full control over the styling using CSS libraries like Tailwind CSS, Bootstrap, Bulma, or custom CSS. This mode supports a pluggable architecture, making it future-proof and flexible.

Pass-Through

The Pass-Through API gives you access to internal DOM elements, allowing you to add custom attributes, events, and styles. This feature eliminates the dependency on the library maintainer for extending APIs, enabling you to customize components as needed. Common use cases include adding test attributes, ARIA attributes, custom events, and additional styling.

Accessibility

PrimeVue complies with WCAG 2.1 AA standards. Each component includes documentation on keyboard and screen reader support. Accessibility experts continuously provide feedback through GitHub and Discord, helping to enhance these features. You can view the accessibility guide for more details.

Add-Ons

PrimeVue offers optional add-ons for those who need additional resources, such as a Figma UI Kit,

premium application templates, and reusable UI blocks called PrimeBlocks. These add-ons are optional, and there is no paywall for using PrimeVue.

Introduction to PrimeVue DataTable

PrimeVue is a comprehensive UI library for Vue.js that provides a wide range of components to enhance web applications. One of its standout components is the DataTable, which offers a rich set of features for displaying and managing tabular data efficiently.

What is a DataTable?

A DataTable is an interactive table that allows users to manipulate and analyze data. Unlike standard HTML tables, DataTables comes with advanced functionalities like sorting, filtering, pagination, and more, making it easier to handle large datasets.

Why Use PrimeVue DataTable?

- **Feature-Rich**: PrimeVue DataTable includes built-in features such as sorting, filtering, pagination, and row selection.
- **Customizable**: It allows extensive customization through templating and offers various styling options to match the look and feel of your application.
- **Responsive**: Designed to be responsive out of the box, ensuring a seamless experience across different devices and screen sizes.
- **Performance**: Optimized for performance, PrimeVue DataTable can handle large datasets efficiently without compromising on speed or usability.

Key Features

- **Sorting**: Users can sort data by clicking on column headers. Multi-column sorting is also supported.
- **Filtering**: Data can be filtered based on various criteria, allowing users to quickly find the information they need.
- **Pagination**: DataTables support pagination, enabling the display of large datasets in manageable chunks.
- **Row Selection**: Users can select single or multiple rows, with the ability to perform actions on the selected data.
- **Column Resizing and Reordering**: Columns can be resized and reordered to better suit the user's needs.
- **Responsive Layout**: The layout adapts to different screen sizes, ensuring usability on both desktop and mobile devices.



You can find more details via the link: https://primevue.org/datatable/

Activity 4: Install PrimeVue into the Library Web Application

4.1 Open the terminal within our library web application, and run the following code:

```
npm install primevue
npm install @primeuix/themes
```

4.2 Open `main.js` and configure **PrimeVue** to use a theme like Aura from **PrimeUIX**, as shown below

```
import { createApp } from 'vue'
import App from './App.vue'

import 'bootstrap/dist/css/bootstrap.min.css'

import PrimeVue from 'primevue/config'
import Aura from '@primeuix/themes/aura'

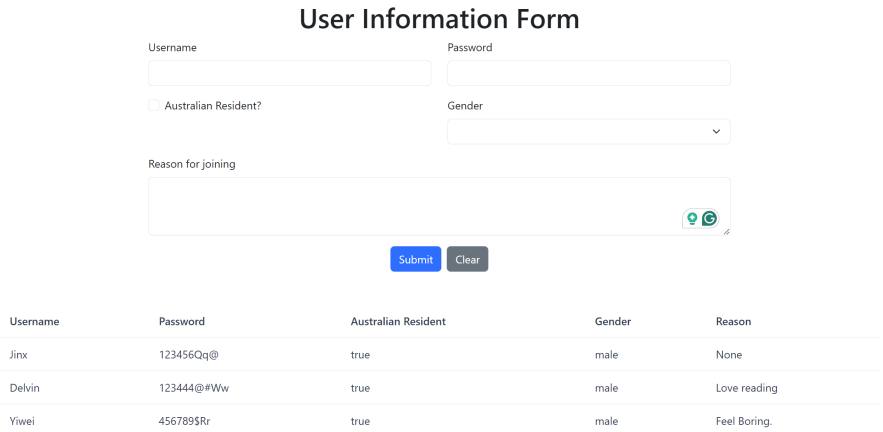
const app = createApp(App);
app.use(PrimeVue, {
  theme: {
    preset: Aura
  }
})

createApp(App).mount('#app')
```

```
import DataTable from 'primevue/datatable';
import Column from 'primevue/column';
import ColumnGroup from 'primevue/columngroup';   // optional
import Row from 'primevue/row';                   // optional
```

https://primevue.org/datatable/

4.3 The goal of this activity is to add a user record to the **PrimeVue DataTable** after the user submits the form, as demonstrated below:



Updated by Russel August 2026.

Don't worry if you are getting an error message for PrimeUI like the below. Use last free, open-Source version MIT-licensed version (v4.0.0) in your `package.json` or apply for non-commercial community license (Core UI libraries at no cost but renew every year).



Post-Lab Advanced Tasks: Industry Taster (60 mins)

Some extra activities for those looking to push themselves beyond the content of this unit.

These tasks are:

1. **NOT** Assessed
2. Only for students seeking extension activities

Credit: Kent Hou Man (Tutor, FIT5032 & Software Engineer, Action Lab)